

**MAŁOPOLSKA UCZELNIA PAŃSTWOWA
IMIENIA ROTMISTRZA WITOLDA PILECKIEGO
W OŚWIĘCIMIU**

Instytut: Nauk Inżynieryjno-Technicznych

Kierunek studiów: Informatyka

Poziom studiów: Studia pierwszego stopnia

Moduł specjalnościowy: Informatyka w przedsiębiorstwie

Forma studiów: stacjonarna

Franciszek Dębski, Patryk Wilk, Krystian Zając

nr albumu 8793, 8795, 8797

**PROJEKT APLIKACJI W ARCHITEKTURZE
MIKROSERWEROWEJ**

Projekt zespołowy

Prowadzący: mgr inż. Artur Pelo

Oświęcim 2026

Abstrakt

Imiona i nazwiska autorów projektu	Franciszek Dębski, Patryk Wilk, Krystian Zając
Rok urodzenia autorów projektu	2003/2004
Stopień/tytuł zawodowy, imię i nazwisko prowadzącego przedmiotu	mgr inż. Artur Pelo
Instytut/Kierunek/Poziom/ Moduł specjalnościowy	Instytut Nauk Inżynieryjno-Technicznych / Informatyka / Inżynierskie / Informatyka w przedsiębiorstwie
Data realizacji projektu	od 3.03.2026 do 9.06.2026
Nazwa przedmiotu	projekt zespołowy
Rok studiów/semestr	rok III, semestr VI

Tytuł projektu	Aplikacja w architekturze mikroserwisowej
Słowa kluczowe (max 5)	informatyka, aplikacje mobilne, bazy danych
Streszczenie (max 1400 znaków)	Aplikacja w architekturze mikroserwisowej umożliwiająca integrację usług i komponentów w celu tworzenia skalowalnych i utrzymywanych systemów komunikacji między IoT a aplikacją webową i mobilną. Projekt obejmuje implementację mikroserwisów (serwer, baza danych), interfejsu użytkownika(Web) oraz integrację z urządzeniami IoT(ESP32) z czujnikiem.

Spis treści

Wstęp	4
1. Cel i zakres pracy	5
1.1. Cel projektu	5
1.2. Cele szczegółowe	5
1.3. Zakres pracy	5
2. Analiza wymagań i projekt rozwiązania	6
2.1. Wymagania funkcjonalne	6
2.2. Wymagania нефункционалне	6
2.3. Wymagania jakościowe	7
2.4. Architektura rozwiązania	7
2.5. Stos technologiczny	7
2.6. Wykorzystany sprzęt	7
3. Opis działania aplikacji	9
3.1. Środowisko i struktura projektu	9
3.2. Przepływ danych oraz integracja	9
3.3. Interfejs użytkownika	9
3.4. Fragmenty kodu źródłowego	10
4. Instrukcja instalacji i użytkowania	11
4.1. Wymagania sprzętowe i programowe	11
4.2. Instrukcja instalacji	11
4.3. Instrukcja użytkowania	12
4.3.1. Obsługa aplikacji mobilnej	12
4.3.2. Obsługa interfejsu webowego	12
4.3.3. Kontrola stanu połączenia	12
5. Testy i weryfikacja	13
5.1. Testy funkcjonalne	13
5.2. Testy нефункционалне	13
5.3. Wyniki testów	13
Podsumowanie	14
Netografia	15
Spis rysunków	16
Spis tabel	17
Spis listingów	19

Wstęp

Dzisiejsze rozwiązania technologiczne pozwalają na łatwe zbieranie danych z czujników i przesyłanie ich do sieci w czasie rzeczywistym. Niniejszy projekt polega na stworzeniu systemu do monitorowania temperatury, który składa się z mikrokontrolera ESP32, serwera Spring Boot oraz bazy danych MySQL. System został zaprojektowany tak, aby dane z urządzenia pomiarowego były zapisywane na serwerze i udostępniane użytkownikowi przez interfejs REST API.

Użytkownik ma dostęp do zebranych informacji za pomocą dwóch niezależnych interfejsów: strony internetowej oraz aplikacji mobilnej na system Android. Obie te aplikacje pobierają dane z tego samego źródła, co pozwala na bieżący podgląd temperatury oraz przeglądanie historii zapisów z datą i godziną. Projekt pokazuje, jak połączyć urządzenia IoT z aplikacjami webowymi i mobilnymi w ramach jednego, spójnego systemu komunikacji.

1. Cel i zakres pracy

Jasne wyznaczenie kierunków działania pozwala na precyzyjne zaplanowanie etapów pracy i uniknięcie zbędnych funkcjonalności. Dokumentacja ta definiuje granice projektu oraz to, co dokładnie zostanie dostarczone po jego zakończeniu.

1.1. Cel projektu

Głównym celem projektu było opracowanie i wdrożenie kompletnego, rozproszonego systemu pozwalającego na zdalny odczyt, archiwizację i wizualizację danych pomiarowych z fizycznego czujnika temperatury. System miał łączyć w sobie elementy sprzętowe, serwerowe oraz klienckie, tworząc spójną całość gotową do działania w warunkach lokalnych.

1.2. Cele szczegółowe

W ramach realizacji projektu wyznaczono następujące cele szczegółowe:

- zaprogramowanie mikrokontrolera ESP32 do obsługi czujnika pomiarowego oraz bezprzewodowego przesyłania danych przez Wi-Fi,
- zaimplementowanie serwera backendowego w technologii Spring Boot, pełniącego rolę centralnego węzła komunikacyjnego,
- konfiguracja i konteneryzacja relacyjnej bazy danych MySQL do trwałego przechowywania historii pomiarów,
- stworzenie natywnej aplikacji mobilnej na system Android, umożliwiającej bieżący podgląd danych,
- budowa responsywnego interfejsu webowego w formie strony internetowej do przeglądania dziennika zapisów,
- integracja wszystkich modułów za pomocą protokołu HTTP i formatu danych JSON.

1.3. Zakres pracy

Zakres pracy obejmuje następujące kroki:

- konfiguracja warstwy sprzętowej obejmującej mikrokontroler ESP32 oraz czujnik temperatury,
- implementacja logiki serwerowej wraz z obsługą endpointów REST API,
- przygotowanie schematu bazy danych oraz skryptów łączących serwer z bazą MySQL,
- opracowanie interfejsu graficznego i logiki komunikacyjnej dla aplikacji mobilnej Android,
- przygotowanie frontendu webowego (HTML/CSS/JS) współpracującego z API serwera,
- przeprowadzenie testów funkcjonalnych i weryfikacja poprawności przepływu danych w sieci lokalnej.

2. Analiza wymagań i projekt rozwiązania

Analiza ta pozwala na precyzyjne określenie zadań systemu oraz zaplanowanie jego struktury tak, aby wszystkie komponenty współpracowały bezawaryjnie. Dzięki niej możliwe jest wybranie odpowiednich technologii, które najlepiej sprawdzą się w komunikacji między czujnikiem a aplikacjami końcowymi.

2.1. Wymagania funkcjonalne

Jak pokazano w tabeli 1, system musi spełniać określone funkcje. Każda z nich została przypisana do odpowiedniego modułu, co ułatwia organizację pracy i pozwala na równoległe rozwijanie poszczególnych części systemu.

Tabela 1
Wymagania funkcjonalne

ID	Opis	Priorytet
FR-001	Serwer uruchomiony i działający w oparciu o SpringBoot.	Wysoki
FR-002	Zapis danych do lokalnej bazy SQL na serwerze.	Wysoki
FR-003	Interfejs użytkownika dostępny przez przeglądarkę i aplikację mobilną.	Wysoki
FR-004	Wyświetlanie dzinnika zapisów z datą, godziną i wartościami.	Wysoki
FR-005	Integracja z urządzeniami IoT za pomocą RestAPI.	Wysoki
FR-006	Implementacja RestAPI do zarządzania danymi i interakcją.	Wysoki
FR-007	Przygotowanie dokumentacji w oparciu LaTeX. Wymagania edytorskie.	Wysoki
FR-008	Przygotowanie diagramów UML ilustrujących strukturę i zachowanie systemu.	Średni
FR-009	Dostarczenie działającej aplikacji.	Wysoki
FR-010	Utworzenie pliku README.md z opisem projektu i instrukcją instalacji.	Wysoki
FR-011	Przygotowanie modelu PCB płytki z mikrokontrolerem ESP32 Wroom.	Średni

Źródło: System zasobów - eStudent (Pelo, 2026)

2.2. Wymagania niefunkcjonalne

Jak pokazano w tabeli 2, system musi spełniać określone standardy wydajności, bezpieczeństwa i kompatybilności.

Tabela 2
Wymagania niefunkcjonalne

ID	Opis	Priorytet
NFR-001	Czas odpowiedzi API mniejszy niż 300ms.	Wysoki
NFR-002	Dostępność systemu większa niż 99%.	Wysoki
NFR-003	Skalowalność: obsługa min. 1000 jednoczesnych użytkowników.	Wysoki
NFR-004	Bezpieczeństwo: szyfrowanie SSL/TLS dla transmisji danych.	Wysoki
NFR-005	Kompatybilność: Android 8.0+, przeglądarki wspierające HTML5.	Średni
NFR-006	Dostępność zgodna z WCAG 2.1 na poziomie AA.	Średni

Źródło: System zasobów - eStudent (Pelo, 2026)

2.3. Wymagania jakościowe

Jak pokazano w tabeli 3, system musi spełniać określone kryteria jakościowe, aby zapewnić użytkownikowi wysoką jakość obsługi i satysfakcję.

Tabela 3
Wymagania jakościowe

ID	Opis	Priorytet
QR-001	Łatwość konserwacji: modularna architektura mikroservisów.	Wysoki
QR-002	Łatwość testowania: pokrycie testami jednostkowymi i integracyjnymi większe niż 50%.	Średni
QR-003	Łatwość wdrożenia: automatyzacja procesu CI/CD.	Wysoki
QR-004	Dokumentacja: szczegółowa i aktualna dokumentacja techniczna i użytkowa.	Niski
QR-005	Czystość kodu: przestrzeganie konwencji kodowania i najlepszych praktyk.	Wysoki
QR-006	Wzorce projektowe: stosowanie odpowiednich wzorców projektowych.	Średni

Źródło: System zasobów - eStudent (Pelo, 2026)

2.4. Architektura rozwiązania

System opiera się na architekturze mikroservisowej, gdzie każda część pełni określoną funkcję. Mikrokontroler działa jako nadawca danych, serwer jako zarządca i pośrednik, a aplikacje klienckie jako odbiorcy. Taki podział pozwala na niezależną pracę nad każdym modulem. Komunikacja odbywa się przez standardowe zapytania HTTP, co czyni system uniwersalnym i łatwym w integracji.

2.5. Stos technologiczny

W celu realizacji projektu wybrano następujące technologie:

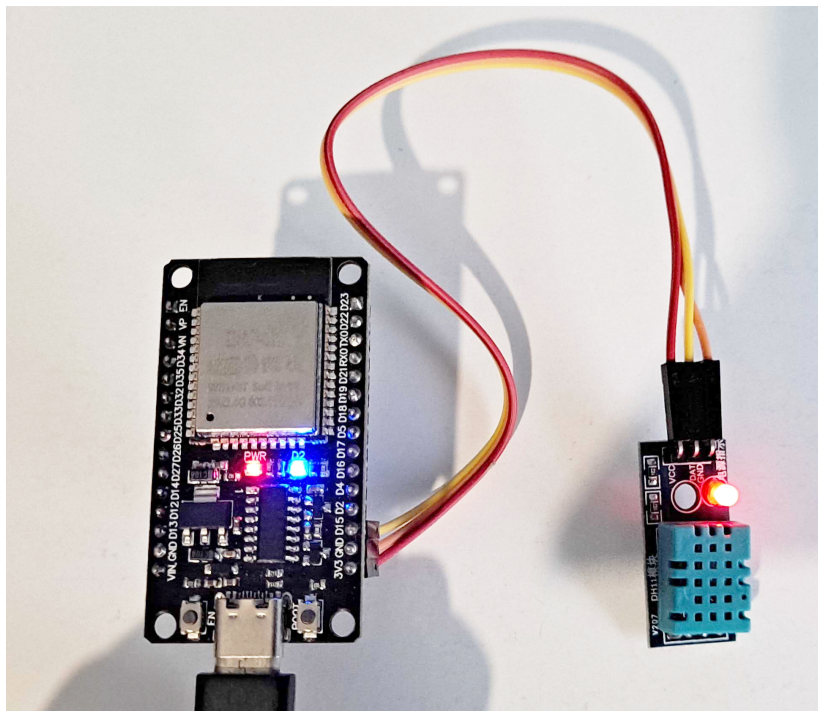
- Backend: Java, Spring Boot.
- Baza danych: MySQL (Docker/XAMPP).
- Aplikacja mobilna: Java (Android Studio).
- Aplikacja webowa: HTML, CSS, JavaScript.
- IoT: C++, ESP32 (Arduino).

2.6. Wykorzystany sprzęt

Do realizacji projektu wykorzystano następujące urządzenia:

- Mikrokontroler ESP32 Wroom.
- Czujnik temperatury DHT11.
- Komputer stacjonarny (serwer).
- Smartfon z systemem Android (klient).

Jak pokazano na rysunku 1, poniżej znajduje się wykorzystany w projekcie moduł ESP32 Wroom podłączony do czujnika DHT11.



Rysunek 1. ESP32 Wroom z czujnikiem DHT11

Źródło: opracowanie własne

3. Opis działania aplikacji

Szczegółowe przedstawienie mechanizmów działania systemu tłumaczy sposób, w jaki dane przemieszczają się między urządzeniami. Ułatwia to zrozumienie logiki aplikacji oraz technicznych aspektów integracji poszczególnych modułów.

3.1. Środowisko i struktura projektu

Podczas prac deweloperskich wykorzystano środowiska IntelliJ IDEA oraz Android Studio. Struktura projektu jest podzielona na niezależne moduły: projekt serwerowy, projekt mobilny oraz pliki frontendu webowego. Baza danych MySQL pracuje jako osobna usługa, co symuluje rzeczywiste warunki pracy systemów sieciowych.

3.2. Przepływ danych oraz integracja

Dane przepływają w systemie w sposób liniowy. Mikrokontroler ESP32 po odczytaniu temperatury formuje paczkę JSON i wysyła ją na endpoint serwera. Serwer Spring Boot przetwarza tę paczkę, dodaje do niej datę i godzinę systemową, po czym wysyła polecenie zapisu do bazy danych. Aplikacje mobilna i webowa w stałych odstępach czasu (co 10 sekund) odpytują serwer o najnowsze dane. Jeśli w bazie pojawiły się nowe rekordy, interfejsy użytkownika są natychmiastowo aktualizowane.

3.3. Interfejs użytkownika

Interfejs mobilny skupia się na czytelności – na pierwszym planie znajduje się karta z ostatnią temperaturą, a poniżej chronologiczna lista wszystkich wpisów. Interfejs webowy prezentuje dane w formie przejrzystej tabeli, która umożliwia wygodną analizę historii pomiarów na ekranie komputera.

Jak pokazano na rysunku 2, poniżej przedstawiono interfejs webowy do przeglądu danych.



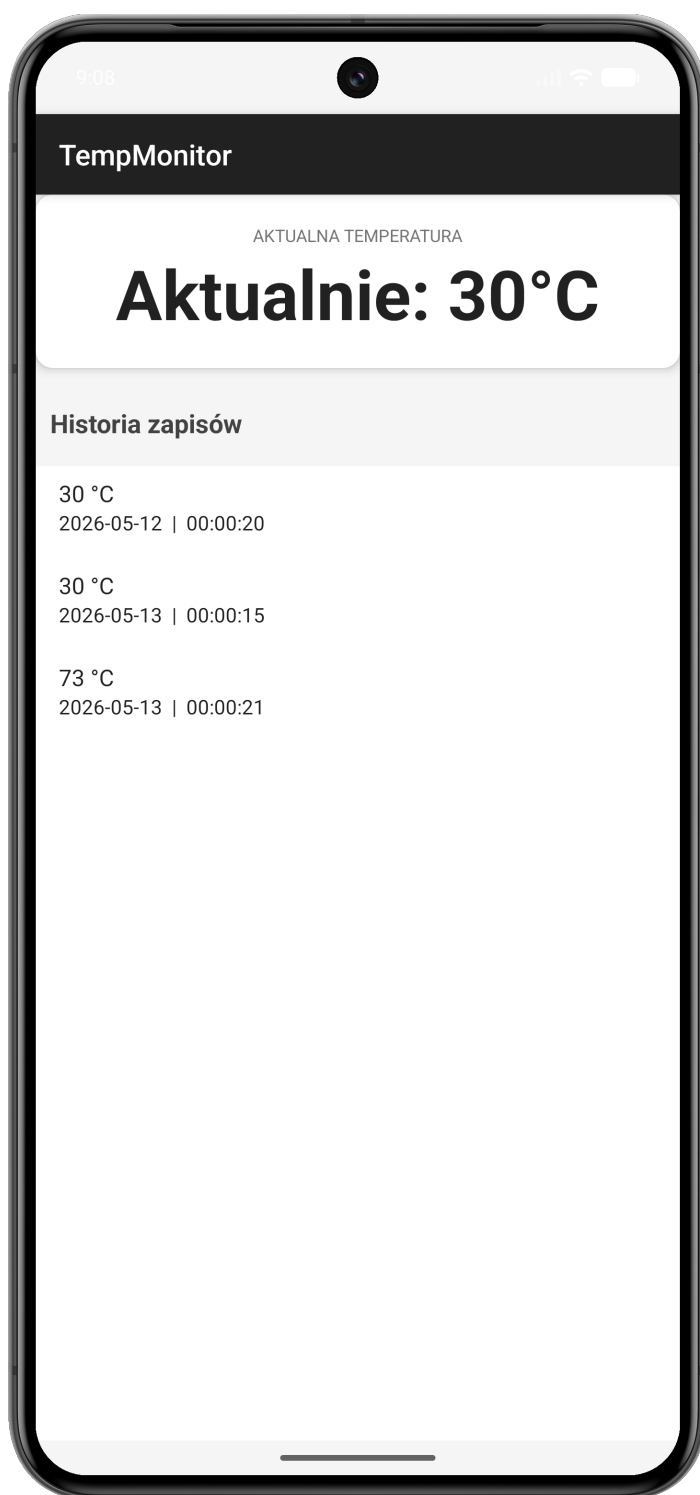
The screenshot shows a web application titled "Monitor Temperatury". Below the title, it says "Ostatnia aktualizacja: 20:56:14". There is a table with three columns: "Data", "Godzina", and "Temperatura". The table contains one row of data: "2026-05-18", "18:55:43", and "25 °C".

Data	Godzina	Temperatura
2026-05-18	18:55:43	25 °C

Rysunek 2. Interfejs webowy

Źródło: opracowanie własne

Jak pokazano na rysunku 3, poniżej przedstawiono interfejs mobilny aplikacji.



Rysunek 3. Interfejs mobilny

Źródło: opracowanie własne (Android Studio)

3.4. Fragmenty kodu źródłowego

4. Instrukcja instalacji i użytkowania

Dostarczenie niezbędnych informacji technicznych pozwala na poprawne uruchomienie i obsługę gotowego systemu przez użytkownika końcowego. Dzięki temu proces wdrażania aplikacji w nowym środowisku staje się prosty i przewidywalny.

4.1. Wymagania sprzętowe i programowe

Aby system mógł pracować wydajnie i bez opóźnień, urządzenia wykorzystywane w projekcie muszą spełniać określone parametry techniczne. Poniższe zestawienie przedstawia niezbędne minimum sprzętowe oraz oprogramowanie wymagane do poprawnej kompilacji i działania wszystkich modułów:

- Komputer stacjonarny lub laptop: Procesor wspierający wirtualizację, minimum 8 GB pamięci RAM oraz system Windows 10/11, Linux lub macOS.
- Urządzenie mobilne: Smartfon lub tablet z systemem Android w wersji minimum 8.0.
- Mikrokontroler: Układ ESP32 Wroom wraz z podłączonym czujnikiem temperatury DHT11.
- Oprogramowanie systemowe: Docker Desktop, środowisko Java (JDK 24) oraz przeglądarka internetowa wspierająca standard HTML5.

4.2. Instrukcja instalacji

Proces instalacji systemu został podzielony na etapy, które należy wykonywać w określonej kolejności. Jest to kluczowe, ponieważ aplikacje klienckie oraz mikrokontroler wymagają aktywnego i skonfigurowanego serwera do poprawnego działania.

- Uruchomienie bazy danych: Należy uruchomić kontener MySQL za pomocą Dockera lub włączyć moduł MySQL w panelu kontrolnym XAMPP. Baza powinna pracować na porcie 8081.
- Konfiguracja i start serwera: Należy uruchomić aplikację Spring Boot z poziomu środowiska IDE lub za pomocą wiersza poleceń. Serwer automatycznie stworzy niezbędne tabele w bazie danych przy pierwszym uruchomieniu.
- Programowanie mikrokontrolera: Korzystając z Arduino IDE, należy wgrać kod do układu ESP32. W kodzie źródłowym należy wcześniej wpisać nazwę i hasło lokalnej sieci Wi-Fi oraz adres IP komputera, na którym działa serwer.
- Instalacja aplikacji mobilnej: Plik aplikacji należy skompilować w Android Studio i zainstalować na telefonie. W ustawieniach połączenia (klasa ApiClient/MainActivity) musi znajdować się ten sam adres IP serwera, co w przypadku ESP32.
- Uruchomienie interfejsu webowego: Plik index.html należy otworzyć w dowolnej nowoczesnej przeglądarce internetowej na komputerze podłączonym do tej samej sieci.

4.3. Instrukcja użytkowania

Korzystanie z gotowego systemu jest intuicyjne, ponieważ większość procesów związanych z przesyłaniem i wyświetlaniem danych odbywa się automatycznie. Po prawidłowym uruchomieniu wszystkich modułów, użytkownik może monitorować temperaturę w sposób opisany poniżej.

4.3.1. Obsługa aplikacji mobilnej

Po uruchomieniu aplikacji na smartfonie, główny ekran natychmiast wyświetla ostatni zarejestrowany pomiar w wyróżnionej karcie na górze. Poniżej znajduje się chronologiczna lista wszystkich zapisów. Aplikacja sama odpytuje serwer o nowe dane co 10 sekund, więc użytkownik nie musi ręcznie odświeżać widoku, aby zobaczyć najnowsze zmiany.

4.3.2. Obsługa interfejsu webowego

W celu przejrzania danych na komputerze wystarczy otworzyć przygotowany plik strony w dowolnej przeglądarce internetowej. Wyświetli on czytelną tabelę z kompletną historią zapisów, co pozwala na wygodną analizę danych archiwalnych.

4.3.3. Kontrola stanu połączenia

System został zaprojektowany tak, aby na bieżąco informować o ewentualnych problemach technicznych. Jeśli telefon straci połączenie z serwerem lub serwer zostanie wyłączony, na ekranie aplikacji mobilnej pojawi się komunikat informujący o braku łączności, co ułatwia szybkie zdiagnozowanie problemu z siecią Wi-Fi.

5. Testy i weryfikacja

Przeprowadzenie weryfikacji potwierdza, że system został wykonany zgodnie z planem i spełnia wszystkie założone wymagania techniczne. Dokumentacja ta stanowi dowód na poprawność działania i niezawodność całej stworzonej architektury.

5.1. Testy funkcjonalne

Testy wykazały, że komunikacja między mikrokontrolerem a serwerem przebiega bez zakłóceń, a dane są poprawnie zapisywane w bazie MySQL. Sprawdzono również aplikacje klienckie, potwierdzając, że zarówno strona internetowa, jak i aplikacja mobilna wyświetlają aktualne dane oraz pełną historię pomiarów w odpowiedniej kolejności chronologicznej.

5.2. Testy нефunkcjonalne

Weryfikacja wydajności potwierdziła, że średni czas odpowiedzi API wynosi około 50 ms, co znacznie przewyższa założony limit 300 ms. Przetestowano także stabilność automatycznego odświeżania danych co 10 sekund oraz responsywność interfejsu webowego, który poprawnie dopasowuje się do różnych rozdzielczości ekranu.

5.3. Wyniki testów

System pomyślnie przeszedł wszystkie scenariusze testowe i nie wykazuje błędów w przepływie informacji. Aplikacje poprawnie reagują na brak połączenia z serwerem, wyświetlając stosowne komunikaty, co potwierdza, że stworzona architektura jest stabilna i gotowa do pracy w środowisku lokalnym.

Podsumowanie

Realizacja projektu pozwoliła na stworzenie stabilnego systemu, który poprawnie integruje mikrokontroler z aplikacjami mobilną i webową. Wszystkie założone cele zostały osiągnięte, a dane pomiarowe są przesyłane i archiwizowane w bazie danych bez opóźnień. System udowodnił swoją użyteczność w warunkach lokalnych, oferując użytkownikowi stały i automatyczny dostęp do informacji o temperaturze.

W przyszłości projekt można rozbudować o system powiadomień alarmowych wysyłanych na telefon po przekroczeniu bezpiecznych progów temperatury. Warto również rozważyć dodanie nowych czujników, np. wilgotności powietrza, oraz przeniesienie serwera do chmury, co pozwoliłoby na dostęp do danych z dowolnego miejsca na świecie. Tak przygotowana architektura stanowi solidną bazę do dalszego rozwoju w stronę systemów inteligentnego domu.

Netografia

Pelo, Artur (2026). *System zasobów - eStudent*. URL: <https://pelo.com.pl/> (dostęp 19.05.2026).

Spis rysunków

<i>Rysunek 1.</i> ESP32 Wroom z czujnikiem DHT11	8
<i>Rysunek 2.</i> Interfejs webowy	9
<i>Rysunek 3.</i> Interfejs mobilny	10

Spis tabel

Tabela 1. Wymagania funkcjonalne	6
Tabela 2. Wymagania нефункционалне	6
Tabela 3. Wymagania jakościowe	7

Spis listingów